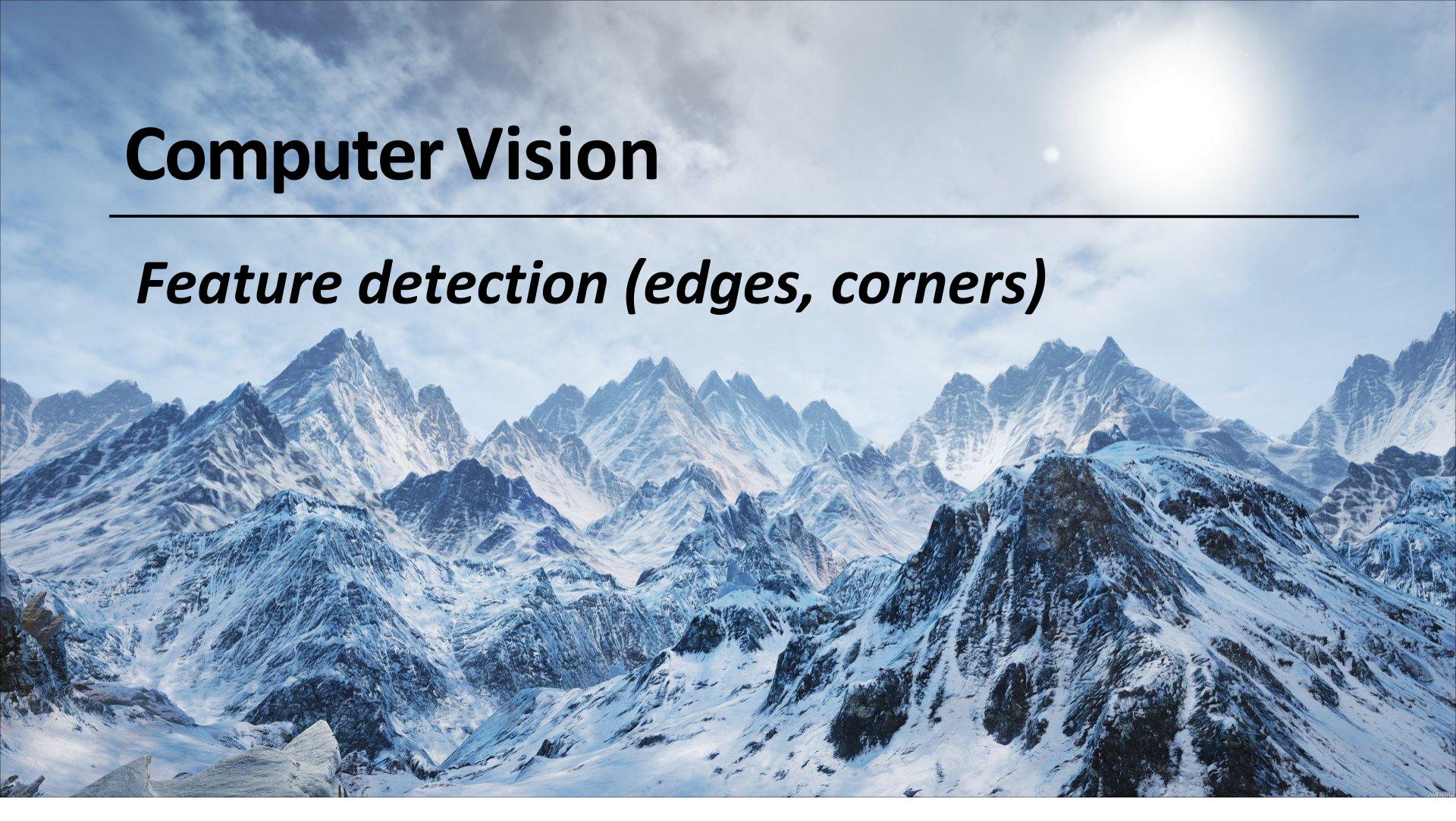
Computer Vision

Feature detection (edges, corners)



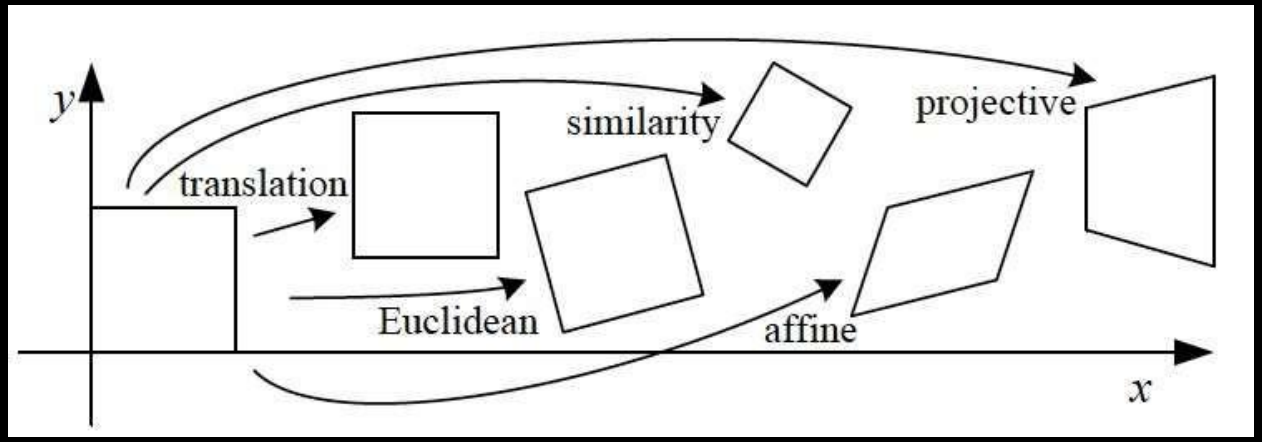
Text resources

Computer Vision: Algorithms and Applications

– Section 7

The basic image point matching problem

- Suppose I have two images related by some transformation. Or have two images of the same object in different positions.



The basic image point matching problem

- Suppose I have two images related by some transformation. Or have two images of the same object in different positions.

The basic image point matching problem

- Suppose I have two images related by some transformation. Or have two images of the same object in different positions.

The basic image point matching problem

- How to find the transformation of image 1 that would align it with image 2?

The basic image point matching problem

- How to find the transformation of image 1 that would align it with image 2?

Image 1

One-point matches to what?

Image 2

Which point? / Which point

The basic image point matching problem

- We need matching corresponding points

We want *Local*(1) *Features*(2)

- Goal: Find points in an image that can be:
  - Found in other images
  - Found precisely – well localized
  - Found reliably – well matched

We want *Local*(1) *Features*(2)

Why?

- Want to compute a fundamental matrix to recover geometry
- Robotics/Vision: See how a bunch of points move from one frame to another. Allows computation of how camera moved -> depth -> moving objects
- Build a panorama…

Suppose you want to build a panorama

M. Brown and D. G. Lowe. Recognising Panoramas. ICCV 2003

Suppose you want to build a panorama

M. Brown and D. G. Lowe. Recognising Panoramas. ICCV 2003

Suppose you want to build a panorama

M. Brown and D. G. Lowe. Recognising Panoramas. ICCV 2003

Suppose you want to build a panorama

M. Brown and D. G. Lowe. Recognising Panoramas. ICCV 2003

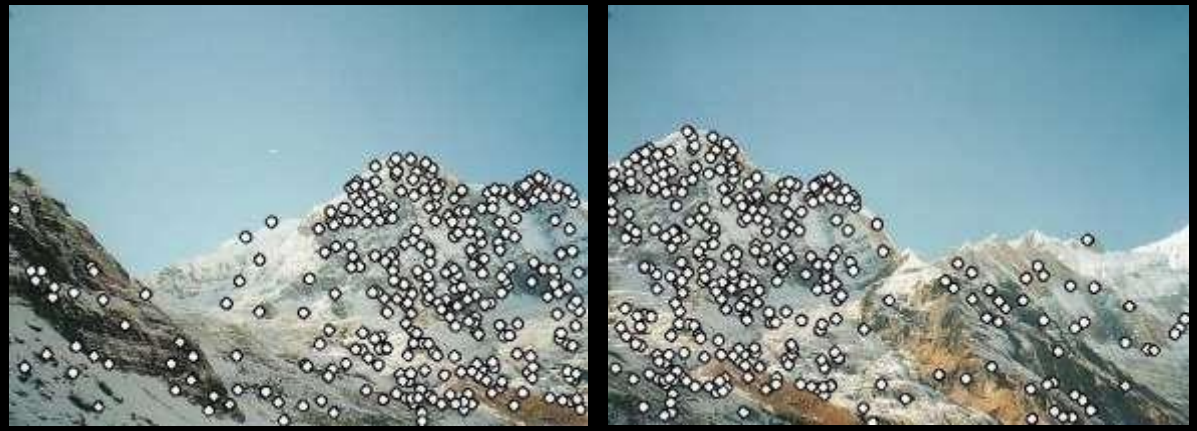
How do we build panorama?

- We need to match (align) images

Matching with Features

STEP 01:

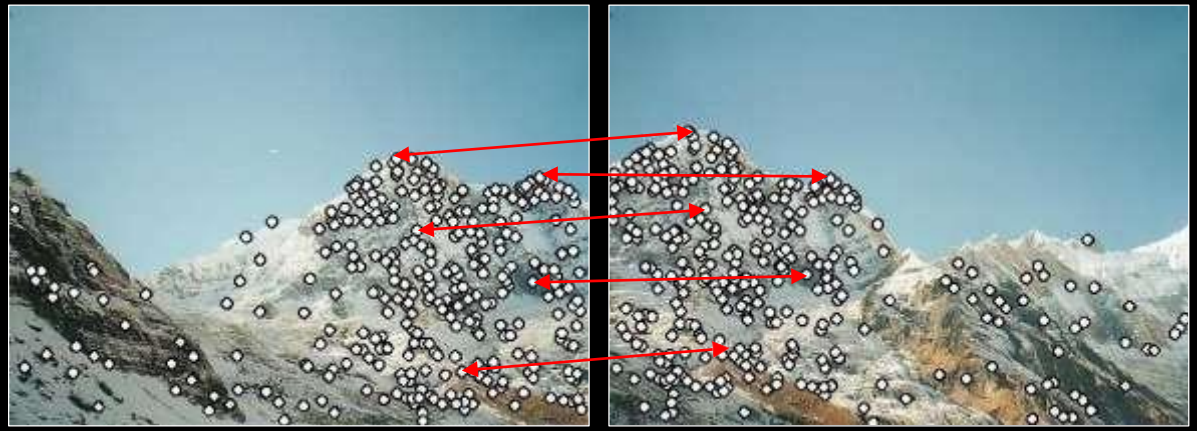
- Detect/ extract features (feature points) in both images



Matching with Features

STEP 02:

- Match features - find corresponding pairs



Matching with Features

Step 03:

- Use these pairs to align images

Activity

Answer questions in Section 1 and Section 2

Tutorial: https://shorturl.at/DTG1D

Matching with Features

Summary

- 1) Extract features
- 2) Match features
- 3) Algin images

Note: These features should be ROBUST against, changes of illumination, occlusions, scale variants

Matching with Features

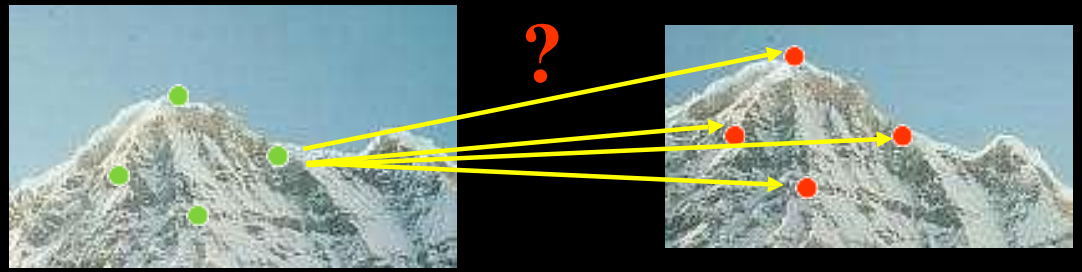
- Problem 1:Detection problem
  - Unable to detect the same point in both images.

no chance to match!

We need a repeatable detector – Algorithm should be consistent enough to detect exact same point even camera/lighting changed

Matching with Features

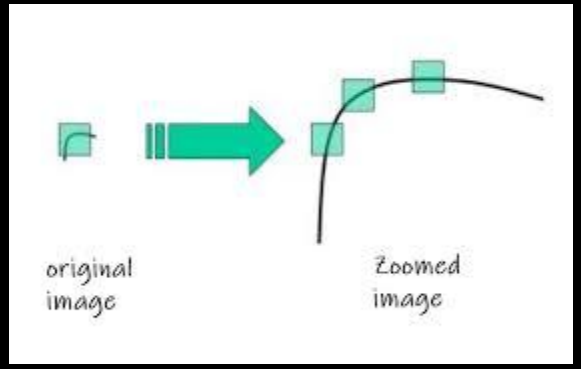
- Problem 2: Recognition Problem
- Correctly linking the points once they are detected



We need a reliable and distinctive ***descriptor*** **– Setting a unique tag for a pixel. Description should be unique enough to stand out from all other points.**

Matching with Features

- Problem 3: Scale Invariance
- Sharp corners looks different depending on how close you are to it.



Scale Invariant Detectors (SIFT)

More motivation…

- Feature points are used also for:
  - Image alignment (e.g. homography or fundamental matrix)
  - 3D reconstruction
  - Motion tracking
  - Object recognition
  - Indexing and database retrieval
  - Robot navigation
  - … other

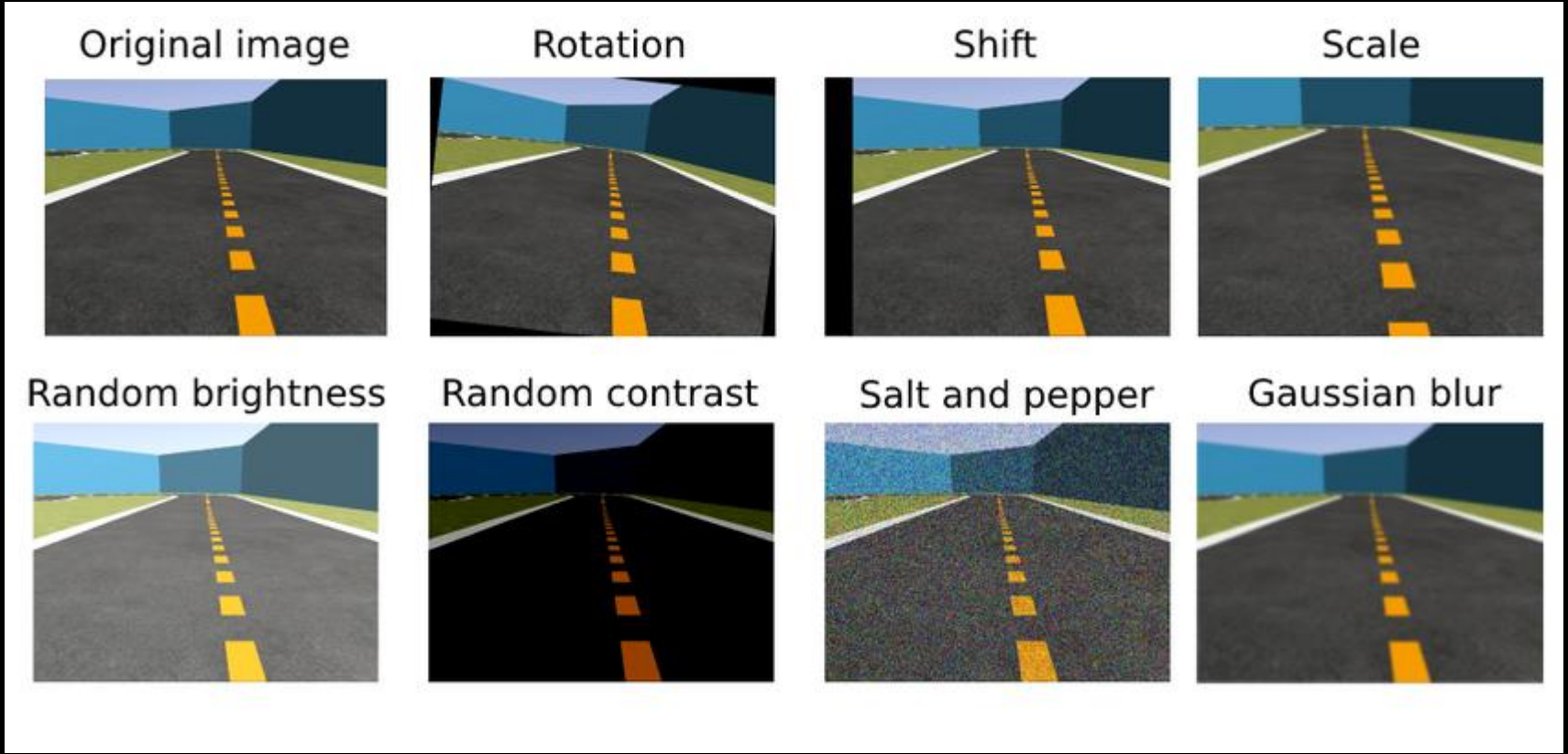
Characteristics of good features

Characteristics of good features

Repeatability/Precision

- The same feature can be found in several images despite geometric and photometric transformations

Characteristics of good features



Characteristics of good features

Saliency/Matchability

- Each feature has a distinctive description. If a feature looks similar to many other points difficult to match

Eg: If the feature looks like a point in a flat texture less region it is difficult to match correctly.

Characteristics of good features

Compactness and efficiency

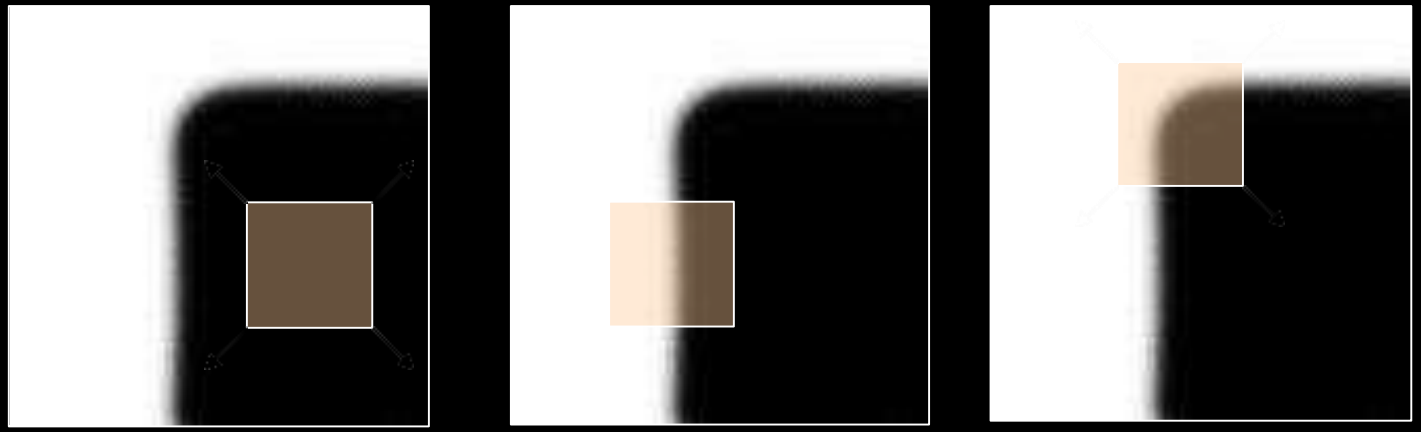
- Should identify far fewer features than there are pixels in the image.
- Allows much faster processing

Characteristics of good features

Locality

- A feature occupies a relatively small area of the image; robust to clutter and occlusion
- Computer can still recognize an object even if parts of it are blocked or background messy.

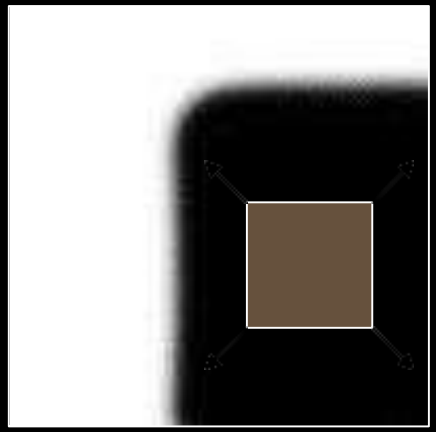
Corner Detection: Basic Idea

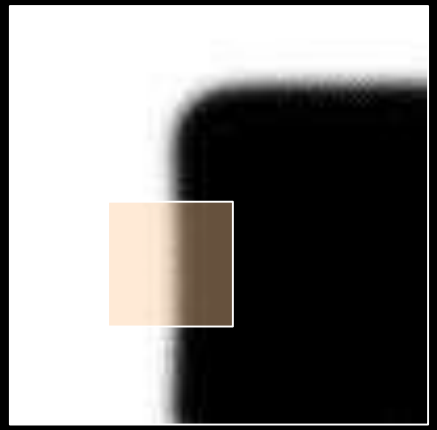


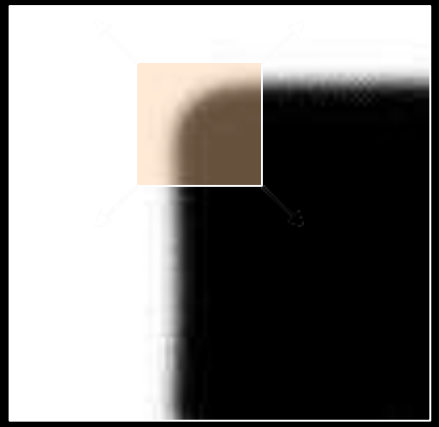
Intensity changes in which direction?

Source: A. Efros

Corner Detection: Basic Idea

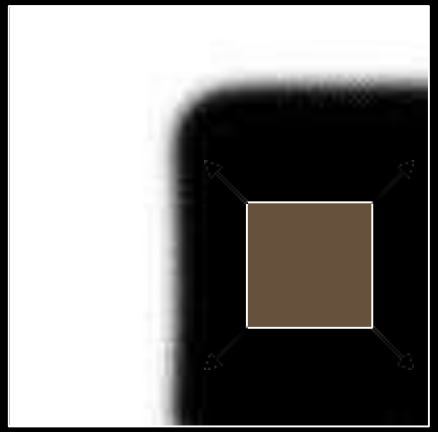




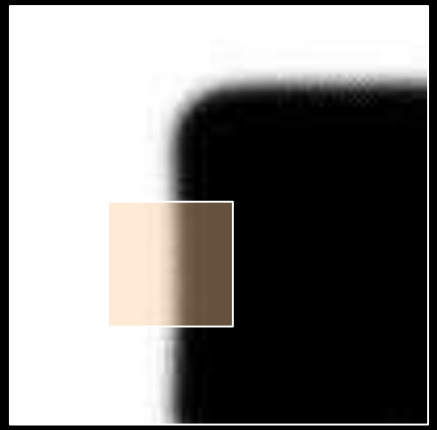


“flat” region: no change in all directions

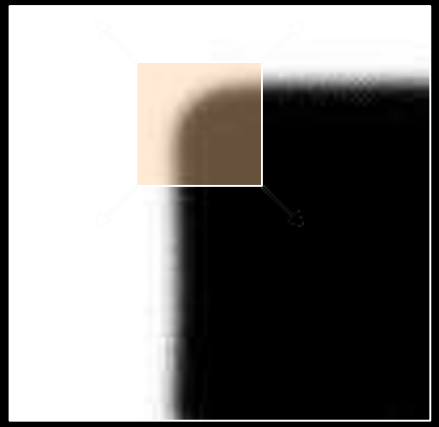
Corner Detection: Basic Idea



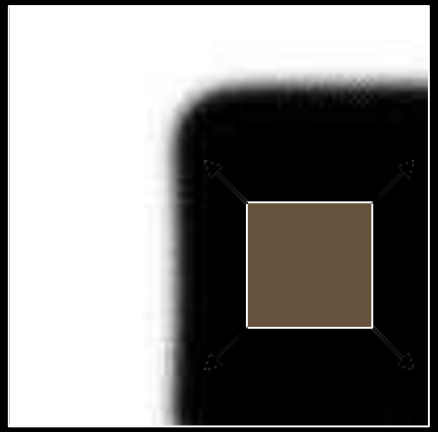
"flat" region:
no change in
all directions



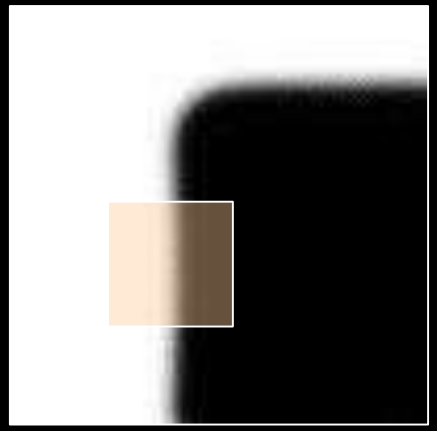
"edge":
no change
along the edge
direction



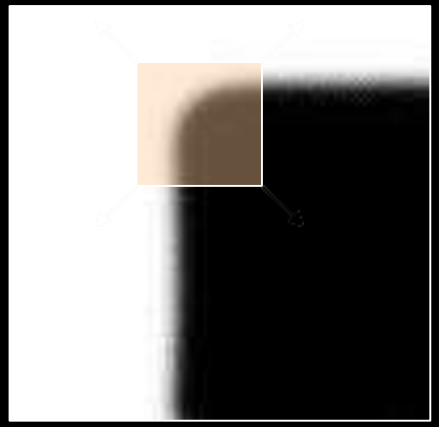
Corner Detection: Basic Idea



"flat" region:
no change in
all directions



"edge":
no change
along the edge
direction

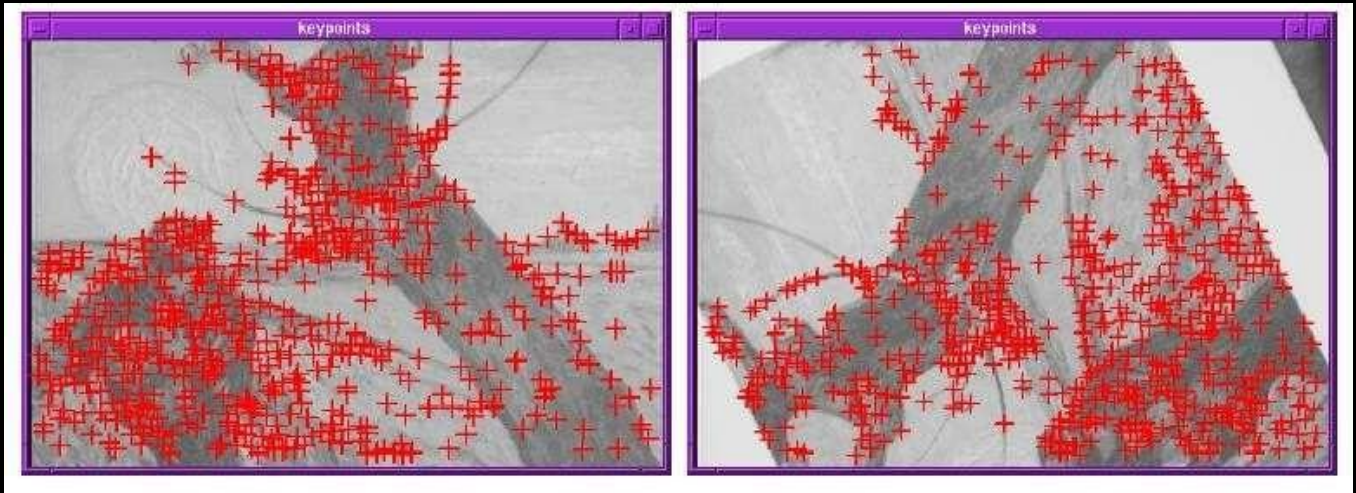


"corner":
significant change
in all directions
with small shift

Source: A. Efros

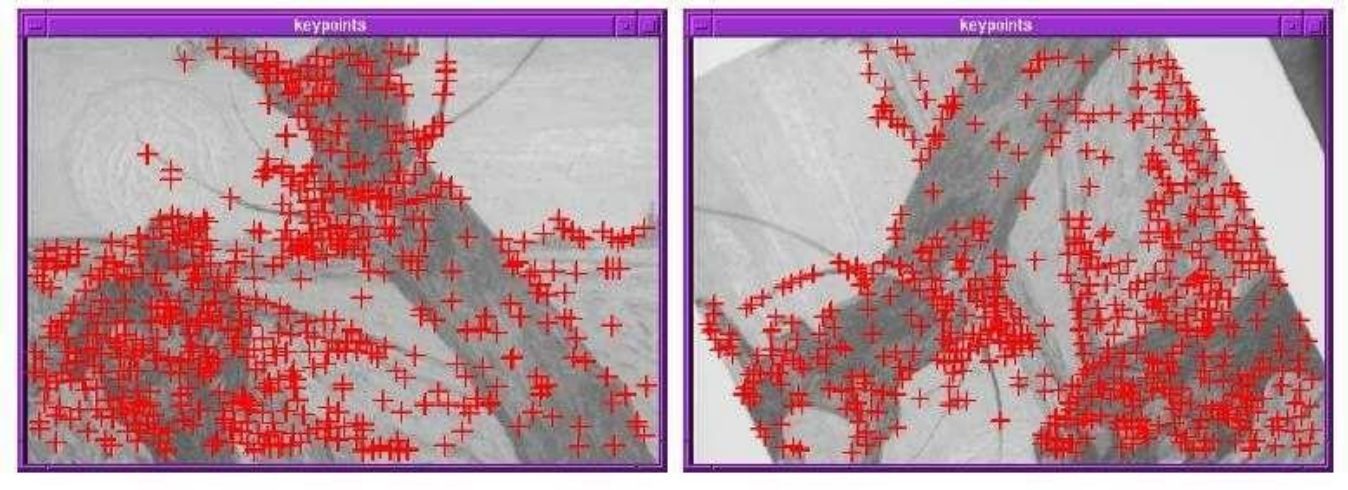
Finding Corners

- Key property: in the region around a corner, image gradient has two or more dominant directions



Finding Corners

C. Harris and M. Stephens. *"A Combined Corner and Edge Detector," Proceedings of the 4th Alvey Vision Conference*: **1988**



Activity

Try the questions up to Section 6

Corner Detection: Mathematics

Based on how much the appearance of a local image window changes when it is shifted by small amount.

Source: R. Szeliski

Corner Detection: Mathematics

Change in appearance for the shift [*u,v*]:

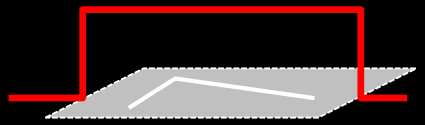
$$E(u,v) = \sum_{x,y} w(x,y)\left[I(x+u, y+v) - I(x,y)\right]^2$$

Window function

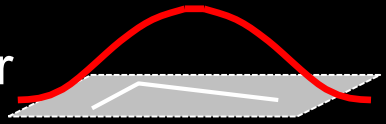
Shifted intensity

Intensity

Window function $w(x,y)$ =



or



1 in window, 0 outside

Gaussian

Source: R. Szeliski

Corner Detection: Mathematics

Change in appearance for the shift [*u,v*]:

$$E(u,v) = \sum_{x,y} w(x,y)\left[I(x+u,y+v) - I(x,y)\right]^2$$

We want to find out how this function behaves for ***small*** shifts (u,v near 0,0)

Corner Detection: Mathematics

Change in appearance for the shift [*u,v*]:

$$E(u,v) = \sum_{x,y} w(x,y)\left[I(x+u,y+v) - I(x,y)\right]^2$$

We want to find out how this function behaves for ***small*** shifts (u,v near 0,0)

Taylor expansion

Corner Detection: Mathematics

Change in appearance for the shift [*u*,*v*]:

$$E(u,v) = \sum_{x,y} w(x,y)\left[I(x+u,y+v) - I(x,y)\right]^2$$

Second-order Taylor expansion of $E(u,v)$ about (0,0) (local quadratic approximation for small u,v):

Corner Detection: Mathematics

We know Taylor Expansion with first order approximation:

$$f(x+u, y+v) \approx f(x, y) + uf_x(x, y) + vf_y(x+y) \Rightarrow I(x+u, y+v) \approx I(x, y) + uI_x(x, y) + vI_y(x+y)$$

Using this we can write

$$E(u, v) = \sum w(x, y) \left[I(x + u, y + v) - I(x, y) \right]^2$$

$$E(u, v) = \sum w(x, y) \left[I(x,y) + uI_x(x,y) + vI_y(x,y) - I(x, y) \right]^2$$

$$E(u, v) = \sum w(x, y) \left[uI_x(x,y) + vI_y(x,y) \right]^2$$

$$E(u, v) = \sum w(x, y) \left[u^2 I_x^2 + v^2 I_y^2 + 2uvI_xI_y \right]$$

Corner Detection: Mathematics

$$E\ (u,v) = \sum_{x,y} w(x,y)\left[u^2 I_x^{\ 2} + v^2 I_y^{\ 2} + 2uvI_xI_y\right]$$

Rewriting this in matrix form

$$M\ = \sum_{x,y} w(x,y)\,[u\ v] \begin{bmatrix} I_x^{\ 2} & I_xI_y \\ I_xI_y & I_y^{\ 2} \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

Corner Detection: Mathematics

$$E\ \ (u,v) = \sum_{x,y} w(x,y)\left[u^2 I_x^{\ 2} + v^2 I_y^{\ 2} + 2uvI_xI_y\right]$$

Rewriting this in matrix form

$$E(u,v) = \sum_{x,y} \quad w(x,y)\,[u\ \ v] \begin{bmatrix} I_x^{\,2} & I_xI_y \\ I_xI_y & I_y^{\,2} \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$E(u,v) = \quad [u\ \ v]\left(\ \sum w(x,y) \begin{bmatrix} I_x^{\,2} & I_xI_y \\ I_xI_y & I_y^{\,2} \end{bmatrix}\ \right) \begin{bmatrix} u \\ v \end{bmatrix}$$

Corner Detection: Mathematics

$$E(u,v) = [u \; v]\left(\sum w(x,y)\begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}\right)\begin{bmatrix} u \\ v \end{bmatrix}$$

M

Corner Detection: Mathematics

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$$

where *M* is a *second moment matrix* computed from image derivatives:

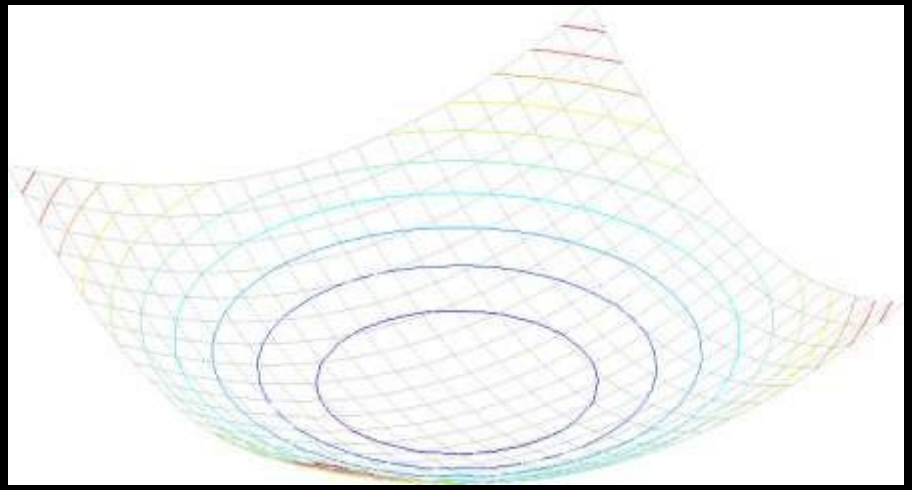
$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Interpreting the second moment matrix

Consider a constant "slice" of $E(u, v)$:

$$\sum_x I^2 u^2 + 2 \sum I_x I_y uv + \sum_y I^2 v^2 = k$$

This is the equation of an ellipse.



Interpreting the second moment matrix

First, consider the axis-aligned case where gradients are either horizontal or vertical

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

If either λ is close to 0, then this is **not** a corner, so look for locations where both are large.

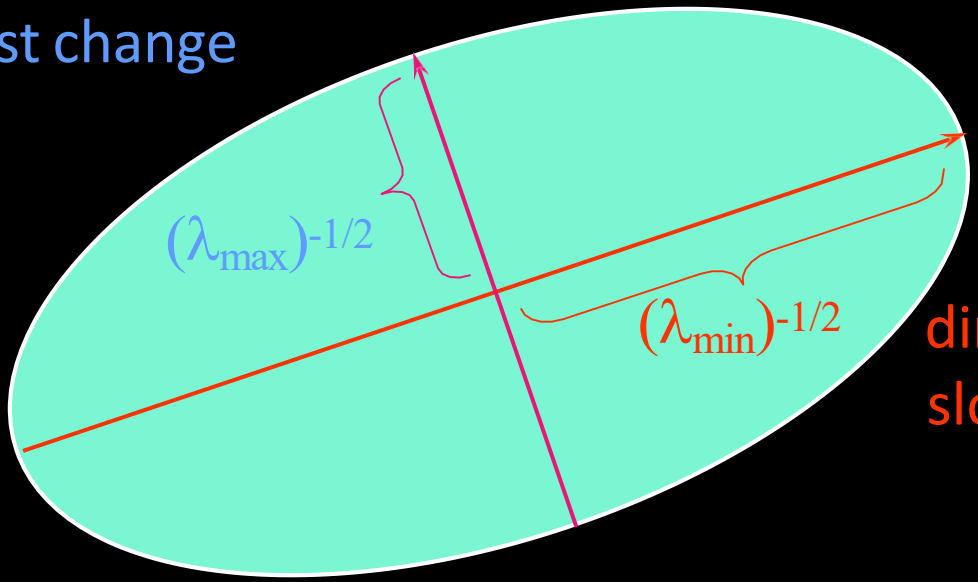
Interpreting the second moment matrix

Diagonalization of M: $M = R^{-1} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R$

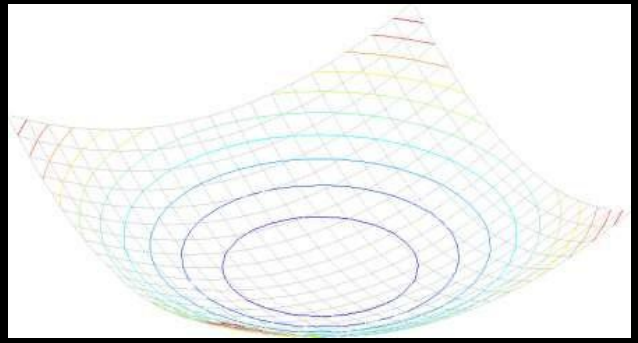
The axis lengths of the ellipse are determined by the eigenvalues and the orientation is determined by *R*

Interpreting the second moment matrix

direction of the fastest change

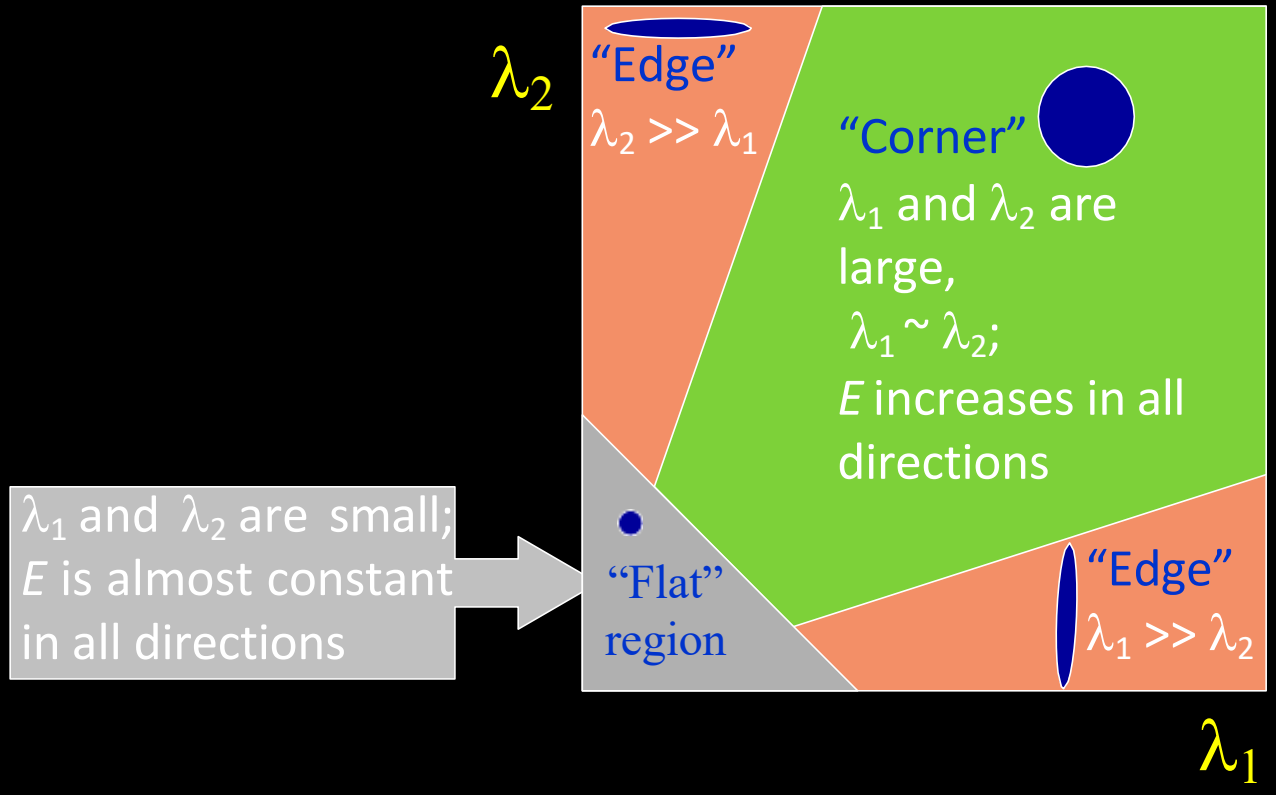


direction of the slowest change



Interpreting the eigenvalues

Classification of image points using eigenvalues of *M*:



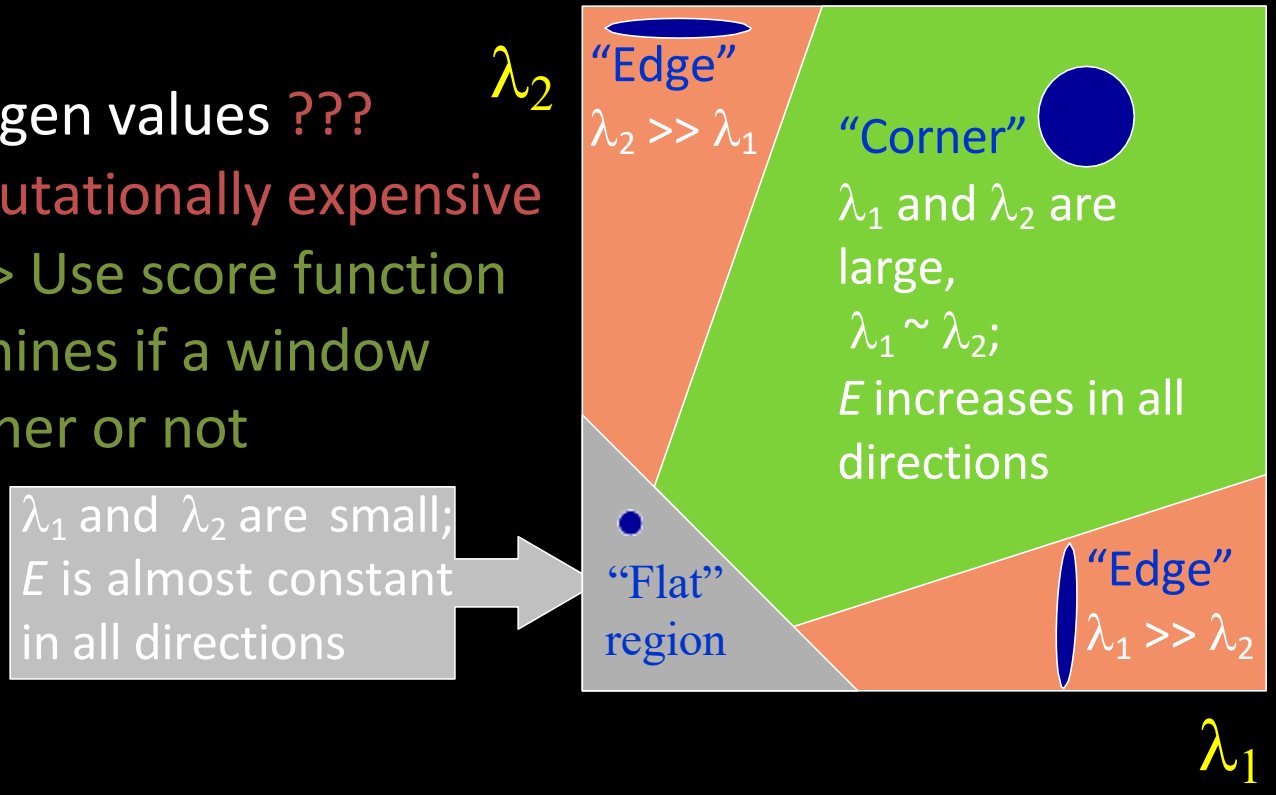
Interpreting the eigenvalues

Classification of image points using eigenvalues of *M*:

Calculating eigen values ???

Computationally expensive

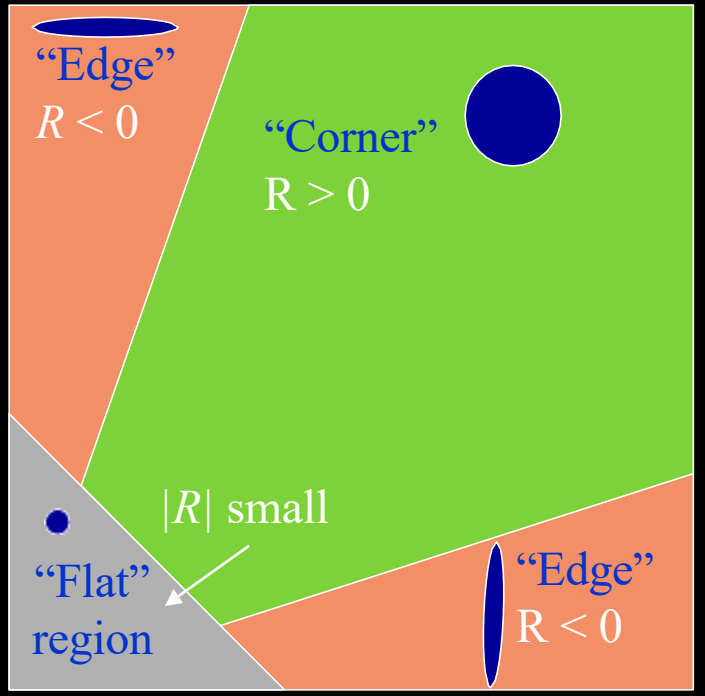
Suggestion => Use score function which determines if a window contain a corner or not



Harris corner response function

$$R = \det(M) - \alpha \, \mathrm{trace}(M)^2$$
$$= \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2$$

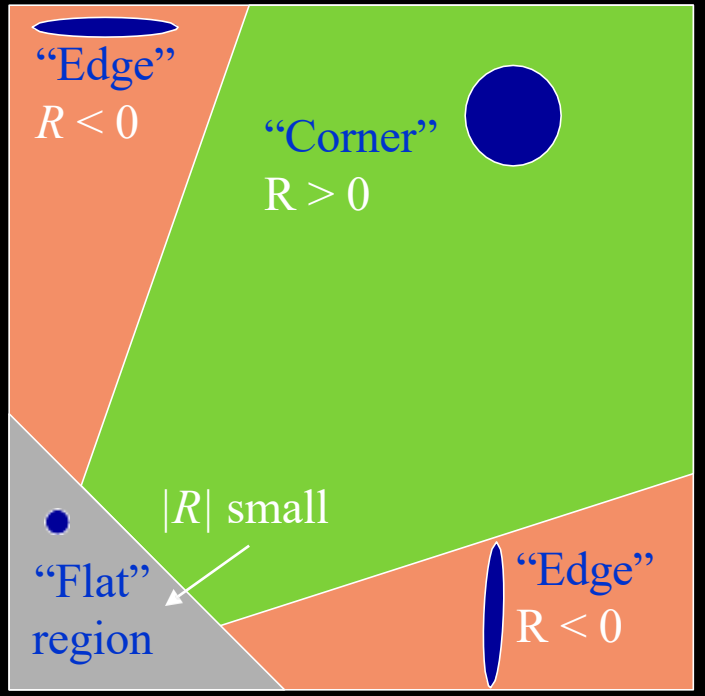
α: constant (0.04 to 0.06)



Harris corner response function

$$R = \det(M) - \alpha \, \mathrm{trace}(M)^2 = \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2$$

R depends only on eigenvalues of M, but don't compute them (no sqrt, so really fast even in the '80s).



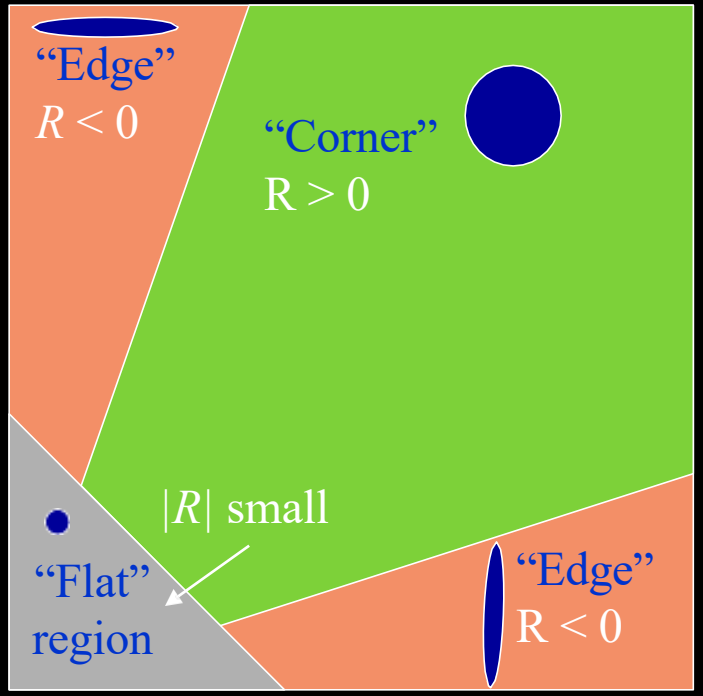
Harris corner response function

$$R = \det(M) - \alpha \, \mathrm{trace}(M)^2 = \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2$$

R is large for a corner

R is negative with large magnitude for an edge

|*R*| is small for a flat region



Harris Detector: Algorithm

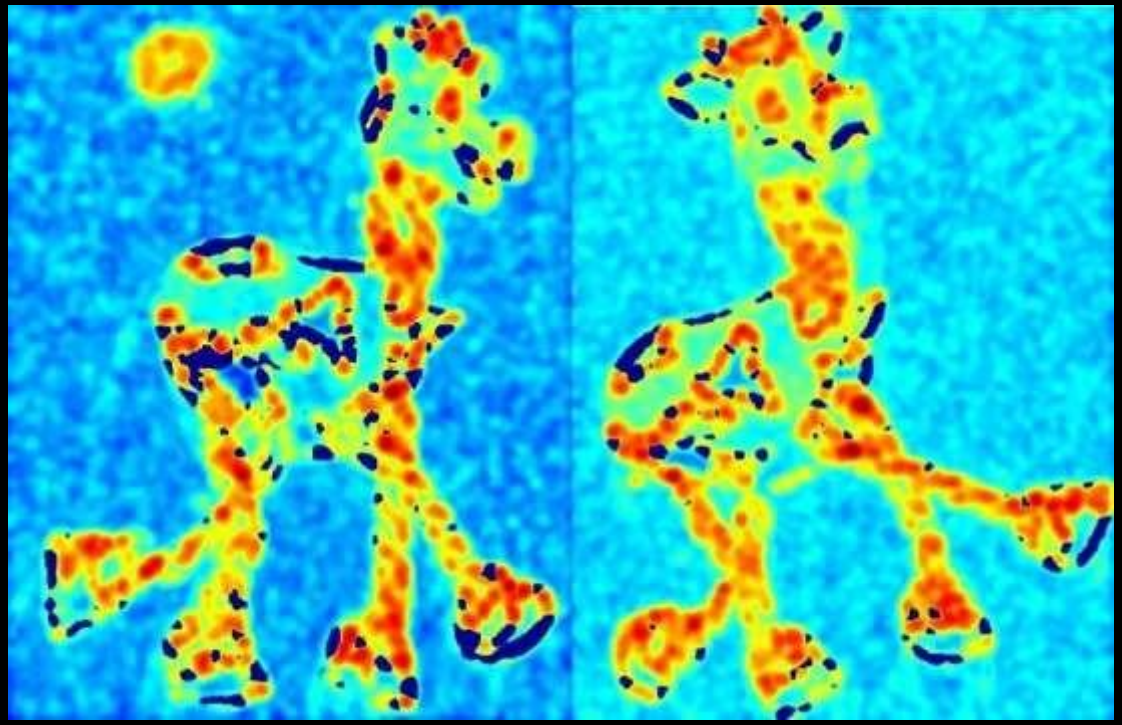
1. Compute Gaussian derivatives at each pixel
2. Compute second moment matrix M in a Gaussian window around each pixel
3. Compute corner response function R
4. Threshold R
5. Find local maxima of response function (nonmaximum suppression)

C. Harris and M. Stephens. "A Combined Corner and Edge Detector.“ *Proceedings of the 4th Alvey Vision Conference*: pages 147—151, 1988.

Harris Detector: Workflow

Harris Detector: Workflow

Compute corner response *R*



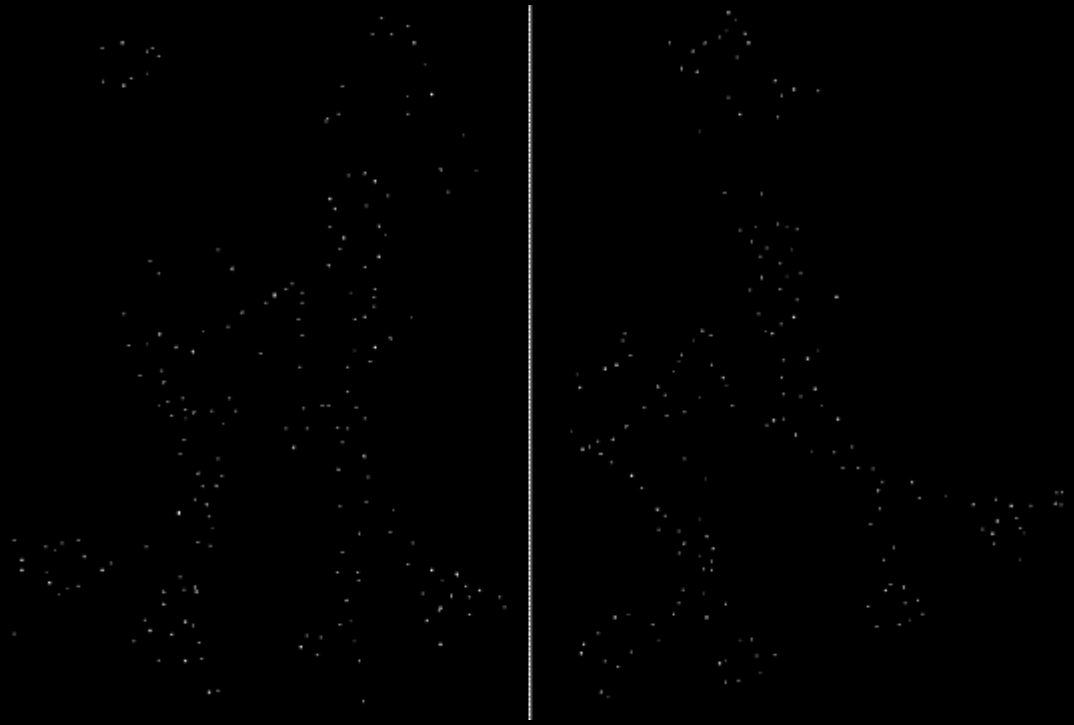
Harris Detector: Workflow

Find points with large corner response: R>threshold

Algorithm gives us a score corresponding to each pixel. We need to do thresholding in order to find corners

Harris Detector: Workflow

Take only the points of local maxima of *R*



Harris Detector: Workflow

Activity:

Try the rest of the questions in the tutorial

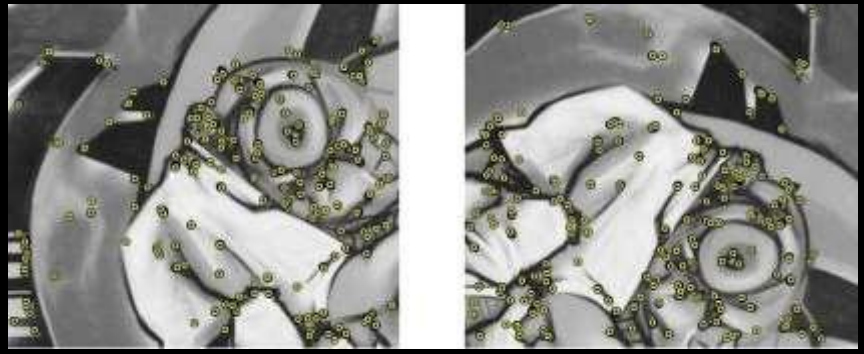
Other corners:

Shi-Tomasi ’94:

“Cornerness” = min (λ_1, λ_2) Find local maximums

cvGoodFeaturesToTrack(...)

Reportedly better for region undergoing affine deformations



Other corners:

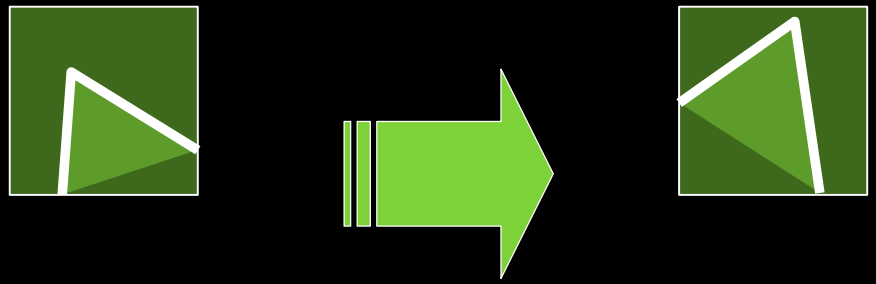
- Brown, M., Szeliski, R., and Winder, S. (2005):

$$\frac{\det M}{\operatorname{tr} M} = \frac{\lambda_0 \lambda_1}{\lambda_0 + \lambda_1}$$

- There are others…

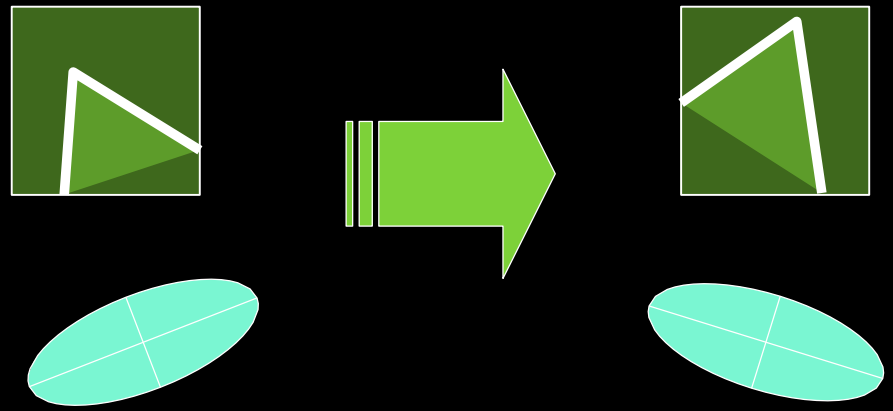
Harris Detector: Some Properties

Rotation invariance?



Harris Detector: Some Properties

Rotation invariance?



Ellipse rotates but its shape (i.e. eigenvalues) remains the same

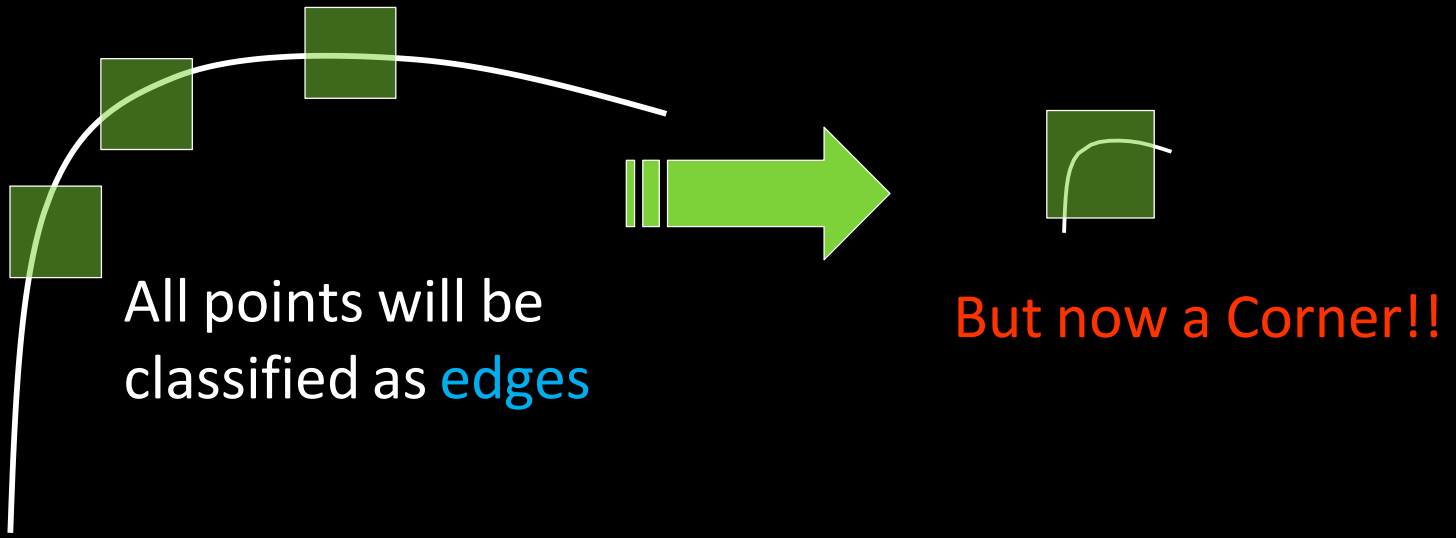
Corner response R is invariant to image rotation

Harris Detector: Some Properties

- Invariant to image scale?

Harris Detector: Some Properties

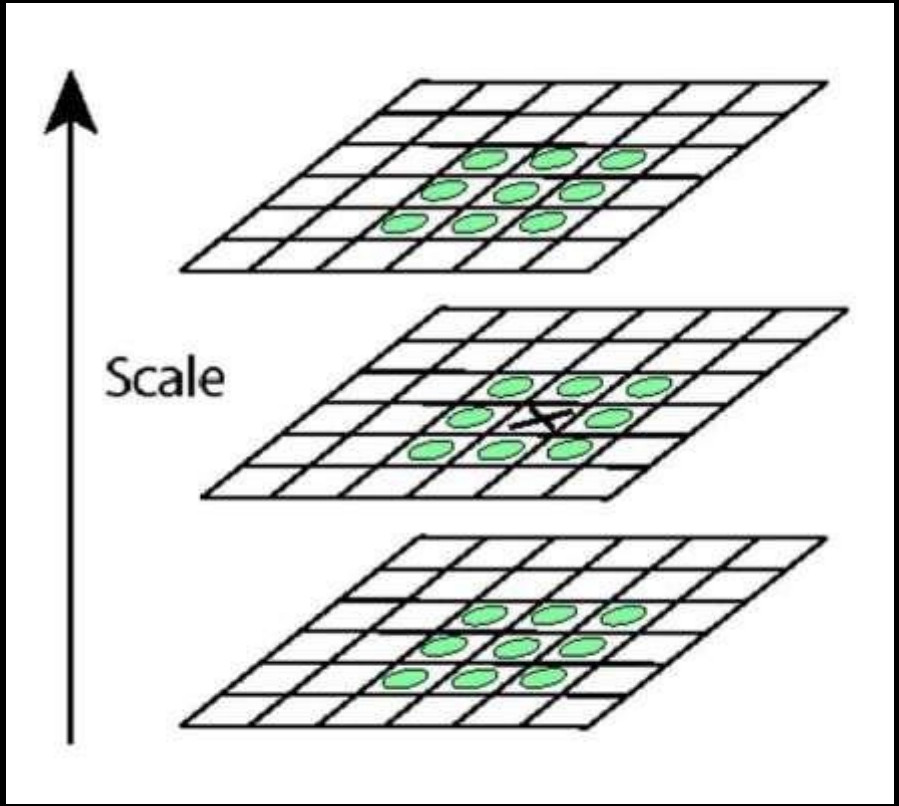
- Not invariant to *image scale*!



Key point localization

- SIFT: **S**cale **I**nvariant **F**eature **T**ransform

- Specific suggestion: use DoG pyramid to find maximum values (remember edge detection?) – then eliminate “edges” and pick only corners.



Implementing Harris Corner Detection

Data set: https://shorturl.at/zMt7M